

LAPORAN PROGRESS MINGGU 3
ANALISIS SPEEDUP SORTING PARALEL MERGE SORT & BITONIC SORT
DENGAN OPENMP



DOSEN PENGAMPU:
SEPTIAN GEGES, S.KOM., M.KOM.

DISUSUN OLEH:
KELOMPOK 2

ELGA EDENIA HAFITZ	2430305030014
GHEA VICTORIA NEISCA	2430305030010
HELEN ANGELINA PUTRI	2430305030034
MEYLIN SAZKIA PUTRI	2430305030004

JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PALANGKA RAYA

2026

1. Status Implementasi Modul

Berikut adalah status seluruh modul yang direncanakan dalam SDD dan kondisinya pada akhir Minggu 3:

Modul	File	Status	Keterangan
MergeSortSeq	merge_sort.cpp	SELESAI	Baseline sequential, semua test PASS
MergeSortPar	merge_sort.cpp	SELESAI	Paralel via omp task untied + cutoff + prefetch
BitonicSortSeq	bitonic_sort.cpp	SELESAI	Baseline sequential, constraint 2^k
BitonicSortPar	bitonic_sort.cpp	SELESAI	Paralel via omp parallel for, 3 schedule + cutoff
DatasetGenerator	dataset_generator.cpp	SELESAI	4 distribusi: random, nearly_sorted, reverse, duplicates
BenchmarkEngine	benchmark.cpp	SELESAI	5 repetisi, filter outlier $\pm 2\sigma$, export CSV
FaultTolerance	fault_tolerance.h	SELESAI	withRetry() + CheckpointManager
UnitTest	test_sorting.cpp	SELESAI	43 test: Smoke/Correctness/EdgeCase/Concurrency/Integration
CSVExporter	benchmark.cpp	SELESAI	Terintegrasi dalam BenchmarkEngine::writeRow()
PlotGenerator	scripts/plot.py	SELESAI	Grafik speedup & efisiensi dari results.csv

Coverage Report

Hasil coverage report yang dijalankan pada akhir Minggu 3:

Modul	Fungsi Tested	Total Fungsi	Coverage
MergeSortSeq	3	3	100%
MergeSortPar	5	5	100%
BitonicSortSeq	4	4	100%
BitonicSortPar	7	7	100%

DatasetGenerator	7	7	100%
BenchmarkEngine	7	7	100%
FaultTolerance	3	3	100%
TOTAL	36	36	100%

Target minimum coverage: 60% — Status: MEMENUHI TARGET (100%)

2. Arsitektur Aktual vs Rencana SDD

Yang Sesuai dengan SDD

- Empat modul utama (DatasetGenerator, Sorter, BenchmarkEngine, Exporter) tetap ada sesuai rencana
- Pipeline utama: DatasetGenerator -> Sorter -> BenchmarkEngine -> CSV tetap sama
- Dua algoritma yang direncanakan (Merge Sort + Bitonic Sort) keduanya selesai diimplementasi
- Jumlah thread yang diuji ({1, 2, 4, 8}) sesuai rencana SDD
- Format output CSV sesuai dengan yang direncanakan di Bab 4 SDD

Yang Berubah dari SDD

- chunk size dynamic: SDD menyebut dynamic schedule dengan chunk=1 (default). Sempat diimplementasikan chunk=64 tanpa justifikasi, kemudian diperbaiki kembali ke default sesuai SDD
- Cutoff Bitonic Sort: SDD tidak menyebut cutoff untuk BitonicSortPar. Setelah analisis, cutoff ditambahkan untuk memenuhi Prinsip 02 Granularitas yang tercantum di materi kuliah
- FaultTolerance module: Tidak ada di SDD awal, ditambahkan di Minggu 3 mengikuti panduan PDF KPT Minggu 3 hal. 7

- d. CSVExporter & PlotGenerator: Di SDD direncanakan sebagai modul terpisah.
Implementasi aktualnya CSVExporter terintegrasi langsung dalam BenchmarkEngine, dan PlotGenerator ada di scripts/plot.py (Python)
- e. UnitTest module: Tidak direncanakan eksplisit di SDD, ditambahkan di Minggu 3 mengikuti panduan KPT

3. Tantangan Teknis dan Solusi

Tantangan	Dampak	Solusi yang Diterapkan
BitonicSortPar tidak punya cutoff threshold	Melanggar Prinsip 02 Granularitas — stage kecil tetap diparalelkan meski overhead > manfaat	Tambah parameter cutoff_ di konstruktor. Stage dengan stageSize < cutoff diproses sequential
dynamic schedule pakai chunk=64 tanpa justifikasi	Tidak konsisten dengan komentar kode sendiri dan penjelasan di laporan SDD	Ubah ke schedule(dynamic) tanpa chunk — default chunk=1 sesuai standar OpenMP
Fase merge di MergeSortPar tidak bisa diparalelkan	Amdahl's Law: ~33% pekerjaan sequential, speedup maks ~3x meski pakai 8 thread	Didokumentasikan sebagai keterbatasan. Parallel merge membutuhkan struktur data tambahan di luar scope
Bitonic Sort butuh data tepat 2^k	Data 1 juta elemen tidak bisa langsung dipakai	padToPow2() pad data ke 1.048.576 (2^{20}) dengan sentinel INT_MAX, lalu removePadding() setelah sort
Implicit barrier di setiap stage Bitonic Sort	Semakin banyak thread, semakin besar potensi overhead	Cutoff threshold mengurangi jumlah stage yang diparalelkan. Stage kecil langsung sequential

	sinkronisasi (~190 barrier untuk $n=2^{20}$)	
Fault handling terbatas untuk shared memory	Crash satu thread di OpenMP = crash seluruh proses, retry tidak bisa memulihkan	withRetry() melindungi dari exception (bukan crash). validateSorted() catat FAIL di CSV

4. Rencana Detail Minggu 4

Target Utama Minggu 4

- Integrasi penuh: jalankan benchmark lengkap dengan data 1 juta elemen, semua kombinasi thread x cutoff x schedule x dataset
- Analisis Amdahl's Law: hitung speedup aktual dari data CSV dan bandingkan dengan prediksi teoritis (maks ~3x untuk 8 thread)
- Grafik performa: generate grafik speedup dan efisiensi menggunakan scripts/plot.py
- Optimasi: identifikasi bottleneck dari profiling, pertimbangkan parallel merge jika waktu memungkinkan
- Laporan akhir: tulis analisis hasil benchmark, simpulan perbandingan Merge Sort vs Bitonic Sort

Jadwal Rencana Minggu 4

Hari	Kegiatan	Penanggung Jawab
Senin	Jalankan benchmark lengkap (data 1 juta, semua kombinasi)	Semua anggota
Selasa	Analisis hasil CSV: speedup, efisiensi, perbandingan antar dataset	Semua anggota
Rabu	Generate dan review grafik dari plot.py	Semua anggota

Kamis	Tulis analisis Amdahl's Law dan kesimpulan	Semua anggota
Jumat	Review akhir, siapkan presentasi, push tag v4.0-week4	Semua anggota

Skenario Pengujian Minggu 4

Benchmark akan menguji kombinasi berikut secara otomatis:

Parameter	Nilai yang Diuji	Jumlah Kombinasi
Dataset type	random, nearly_sorted, reverse, duplicates	4
Thread count	1, 2, 4, 8	4
Cutoff (Merge)	512, 1024, 2048	3
Cutoff (Bitonic)	512, 1024, 2048	3
Schedule	static, dynamic, guided (Bitonic only)	3
Repetisi	5 kali per kombinasi, outlier +/-2sigma dibuang	5
TOTAL kombinasi	4 x 4 x 3 x 3 x 3 x 5	~2160 run